

ScripticX - Learning Platform

Model matematic pentru engine-ul MiniScript+

Document tehnic privind reprezentarea formală, execuția și analiza
complexității limbajului MiniScript+

Realizat de:

Andrei Lascu

Colegiul Național “Gheorghe Munteanu Murgoci”, Brăila

Profesor coordonator:

Prof. Paul Pruş

Colegiul Național “Gheorghe Munteanu Murgoci”, Brăila

Mai 2026

Cuprins

1	Introducere	1
2	Modelul formal al programului	1
3	State-ul intern al interpretorului	1
4	Funcția de tranziție	2
5	Procesul de Tokenizare	3
6	Procesul de parsare și construirea structurii programului	4
7	Reprezentarea expresiilor prin AST	4
8	Evaluarea recursivă a expresiilor	6
9	Despre execuția instrucțiunilor	7
10	Mecanismul de limitare al buclelor infinite	7
11	Complexitatea totală a engineului	8
12	Complexitatea în spațiu	9
13	Despre Analizatorul de Complexitate	9
14	Calcularea scorului de eficiență	10
15	Exemplu de analiză	11
16	Observații privind corectitudinea analizei	12
17	Concluzie	12

1 Introducere

MiniScript+ este un limbaj educațional proiectat pentru a simplifica procesul de învățare a programării. Din punct de vedere matematic, un program MiniScript+ poate fi privit ca o structură formală alcătuită din instrucțiuni, expresii și un state intern care se modifică în timpul execuției programului.

Engine-ul MiniScript+ funcționează precum un interpretor, așadar primește codul sursă, îl transformă într-o reprezentare internă, apoi execută instrucțiunile pas cu pas. Fluxul principal poate fi observat mai jos:

Cod sursă $\longrightarrow$ Tokenizare $\longrightarrow$ Parsare $\longrightarrow$ AST $\longrightarrow$ Evaluare $\longrightarrow$ Output

Această structură este facilă platformei deoarece ne permite atât rularea programului, cât și vizualizarea execuției programului, debugging-ul și estimarea complexității algoritmului.

2 Modelul formal al programului

Un program MiniScript+ poate fi reprezentat ca o secvență finită de instrucțiuni:

$$P = (I_1, I_2, \dots, I_n)$$

unde fiecare instrucțiune I_i aparține unei mulțimi de instrucțiuni admise:

$$I_i \in \mathcal{I}$$

Mulțimea instrucțiunilor poate fi descrisă astfel:

$$\mathcal{I} = \{\text{INPUT, PRINT, ASSIGN, IF, ELSE, WHILE, END}\}$$

Astfel, un program nu este tratat doar ca text, ci ca o listă ordonată de operații care pot modifica state-ul intern al interpretorului.

3 State-ul intern al interpretorului

Deoarece în timpul execuției programului interpretorul nostru trebuie să rețină mai multe lucruri, precum variabilele, ce s-a afișat, linia curentă, câți pași au fost executați și ce input-uri mai sunt de citit, interpretorul păstrează un state intern. Acesta poate fi reprezentat printr-un tuplu:

$$\sigma = (V, O, l, s, Q)$$

unde:

V = tabelul variabilelor definite în program

O = lista valorilor afișate la output

l = linia curentă executată

s = numărul de pași executați

Q = coada valorilor de input

Tabelul variabilelor poate fi privit ca o funcție parțială:

$$V : \text{Identificator} \rightarrow \text{Valoare}$$

De exemplu, dacă în program apare instrucțiunea:

$$X = 5$$

atunci starea variabilelor se modifică astfel:

$$V'(X) = 5$$

iar pentru orice altă variabilă $Y \neq X$:

$$V'(Y) = V(Y)$$

Pe scurt, folosind această metodă, creăm un "snapshot" al runtime-ului curent. :)

4 Funcția de tranziție

Execuția programului poate fi descrisă printr-o funcție de tranziție:

$$\delta(\sigma, I_i) = \sigma'$$

Aceasta înseamnă că, pornind de la o stare curentă σ și executând instrucțiunea I_i , interpretorul obține o stare nouă σ' .

De exemplu, pentru o atribuire de forma:

$$X = expr$$

tranziția poate fi descrisă prin:

$$\delta(\sigma, X = expr) = (V[X \leftarrow eval(expr, V)], O, l + 1, s + 1, Q)$$

unde $eval(expr, V)$ este valoarea expresiei calculate folosind variabilele curente.

Așadar, pentru o instrucțiune de afișare:

PRINT expr

tranziția devine:

$$\delta(\sigma, PRINT\ expr) = (V, O \cup \{eval(expr, V)\}, l + 1, s + 1, Q)$$

Prin urmare, instrucțiunile MiniScript+ pot fi analizate în mod riguros ca fiind transformări asupra unui state intern.

5 Procesul de Tokenizare

Definim **Procesul de Tokenizarea** ca fiind acea etapă în care codul sursă este parcurs caracter cu caracter și împărțit în unități lexicale numite **token-uri**, fiind cunoscut și sub numele de **Lexer**.

Fie următoarea notație a codului sursă printr-un șir de caractere:

$$C = c_1 c_2 \dots c_n$$

atunci, **lexerul** construiește o secvență de token-uri astfel:

$$L(C) = (t_1, t_2, \dots, t_m)$$

unde fiecare token t_i poate fi un identificator, un număr, un string, un operator, o paranteză sau un cuvânt cheie.

Să luăm următorul exemplu:

$$X = X + 1$$

în cazul acesta, secvența de cod devine:

$$(IDENT(X), OP(=), IDENT(X), OP(+), NUMBER(1))$$

Deoarece fiecare caracter este citit cel mult de un număr constant de ori, *complexitatea procesului de tokenizare* este:

$$T_{tokenizare}(n) = O(n)$$

unde n este numărul de caractere din codul sursă.

6 Procesul de parsare și construirea structurii programului

Procesul de parsare verifică dacă token-urile respectă regulile limbajului și construiește reprezentarea (sau modelul) internă a programului.

La nivel simplificat, putem defini sau exprima gramatica limbajului astfel:

$$\begin{aligned} \textit{Program} &\rightarrow \textit{Instr}^* \\ \textit{Instr} &\rightarrow \textit{Assign} \mid \textit{Print} \mid \textit{Input} \mid \textit{If} \mid \textit{While} \\ \textit{Assign} &\rightarrow \textit{id} = \textit{Expr} \\ \textit{Print} &\rightarrow \textit{PRINT Expr} \\ \textit{Input} &\rightarrow \textit{INPUT id} \\ \textit{If} &\rightarrow \textit{IF Expr THEN Program (ELSE Program)? END} \\ \textit{While} &\rightarrow \textit{WHILE Expr Program END} \end{aligned}$$

De asemenea, ținem cont de ordinea operațiilor (prioritatea operatorilor). Așadar, următoarea expresie:

$$X + 2 \cdot Y$$

trebuie interpretată astfel:

$$X + (2 \cdot Y)$$

Ordinea din exemplul de mai jos încalcă rigurozitatea regulilor de calcul la nivel de expresii:

$$(X + 2) \cdot Y$$

Parserul citește token-urile și construiește structuri pentru instrucțiuni, condiții, bucle și expresii. Presupunem că există m token-uri, iar fiecare token este procesat o singură dată sau de un număr constant de ori, atunci:

$$T_{\text{parsare}}(m) = O(m)$$

7 Reprezentarea expresiilor prin AST

Un arbore sintactic abstract, sau AST (*Abstract Syntax Tree*), este o reprezentare ierarhică a structurii logice a codului. Spre deosebire de textul sursă, AST-ul elimină detaliile de scriere ale limbajului și păstrează doar relațiile esențiale dintre operatori, variabile, valori și instrucțiuni. Astfel, expresiile pot fi evaluate corect, respectând prioritatea operatorilor și ordinea de execuție.

Așadar, expresiile din MiniScript+ sunt reprezentate printr-un arbore sintactic abstract (AST):

$$AST_{expr} = (N, E, r)$$

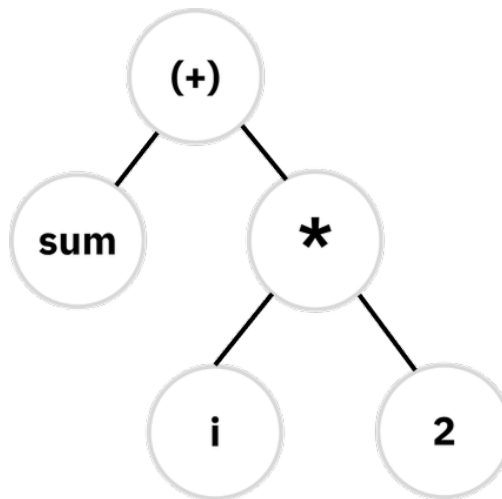
unde:

N = mulțimea nodurilor arborelui
 E = mulțimea muchiilor părinte-copil
 r = rădăcina arborelui

Nodurile interne reprezintă operatori, iar frunzele reprezintă valori sau variabile. De exemplu, pentru expresia:

`sum + i * 2`

AST-ul poate fi reprezentat astfel:

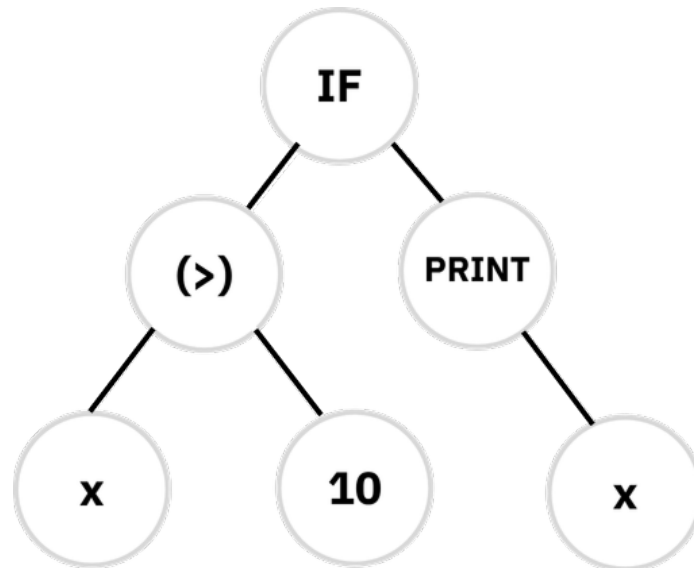


Această reprezentare este importantă deoarece păstrează ordinea corectă de evaluare și prioritatea operatorilor. În exemplul de mai sus, înmulțirea $i \cdot 2$ este evaluată înaintea adunării cu variabila `sum`.

Același principiu se poate aplica și instrucțiunilor de control. De exemplu, o instrucțiune condițională de forma:

`IF X > 10 THEN PRINT X`

poate fi reprezentată printr-un arbore în care nodul principal este instrucțiunea `IF`, iar ramurile sale descriu condiția și blocul executat dacă expresia este adevărată:



În acest caz, nodul IF are **două componente principale**: expresia logică $X > 10$ și instrucțiunea PRINT X. Astfel, folosind un AST putem descrie mai mult decât calcule aritmetice, extinzând reprezentarea până la structura logică a programului.

8 Evaluarea recursivă a expresiilor

În cadrul unui Arbore Sintactic Abstract, evaluarea unei expresii se realizează recursiv pe acesta. Definim următoarea funcție:

$$eval : AST \times V \rightarrow \text{Valoare}$$

Pentru o constantă numerică vom avea:

$$eval(c, V) = c$$

Pentru o variabilă, asemănător:

$$eval(x, V) = V(x)$$

Iar pentru o expresie binară:

$$eval(a \text{ op } b, V) = eval(a, V) \text{ op } eval(b, V)$$

Să luăm următorul exemplu:

$$eval(X + 2 \cdot Y, V) = V(X) + 2 \cdot V(Y)$$

Dacă AST-ul unei expresii are k noduri, fiecare nod este vizitat o singură dată, deci:

$$T_{evaluate_expr}(k) = O(k)$$

De asemenea, spațiul suplimentar folosit de evaluarea recursivă depinde în mod direct de înălțimea arborelui:

$$S_{recursie} = O(h)$$

unde h reprezintă înălțimea AST-ului.

9 Despre execuția instrucțiunilor

După parsare, programul este executat instrucțiune cu instrucțiune. Să presupunem că notăm prin p numărul de instrucțiuni efectiv executate, atunci:

$$T_{executie} = O(p)$$

OBS: Valoarea lui p nu este întotdeauna egală cu numărul de linii din program, deoarece buclele pot executa aceeași instrucțiune de mai multe ori.

Așadar, pentru o buclă cu verificare anterioară de tipul:

$$WHILE\ cond\ DO\ B$$

și blocul B are costul T_B , iar condiția este adevărată de q ori, atunci costul total este aproximativ:

$$T_{while} = O(q \cdot (T_{cond} + T_B))$$

În schimb, pentru două bucle imbricate, cu q_1 și q_2 iterații:

$$T_{nested} = O(q_1 \cdot q_2 \cdot T_B)$$

Iar dacă ambele depind de aceeași dimensiune n , obținem:

$$T_{nested} = O(n^2)$$

10 Mecanismul de limitare al buclelor infinite

Atunci când vorbim despre un limbaj de programare, un aspect important este modul în care acesta previne erorile din timpul runtime-ului, care pot fi și critice, în funcție de situație/context. Pentru siguranță, engine-ul MS+ folosește o limită maximă de pași:

$$s \leq MAX_STEPS$$

La fiecare instrucțiune executată, contorul s crește:

$$s' = s + 1$$

Așadar, dacă condiția:

$$s > MAX_STEPS$$

este adevărată, atunci execuția este oprită și programul este considerat potențial blocat într-o buclă infinită. Desigur, această măsură nu demonstrează matematic existența unei bucle infinite, dar previne blocarea aplicației în browser, limitând utilizatorul și avertizându-l atunci când o astfel de execuție este întâlnită.

11 Complexitatea totală a engineului

În final, procesarea completă a unui program MiniScript+, așa cum am văzut, include mai multe subprocese: tokenizare, parsare, construire AST și execuție:

$$T_{total} = T_{tokenizare} + T_{parsare} + T_{AST} + T_{executie}$$

Așadar, prin înlocuire:

$$T_{total} = O(n) + O(m) + O(k) + O(p)$$

deci:

$$T_{total} = O(n + m + k + p)$$

unde:

n = numărul de caractere din codul sursă

m = numărul de token-uri generate

k = numărul de noduri din AST

p = numărul de instrucțiuni executate

OBS: Dacă toate aceste mărimi sunt raportate la dimensiunea totală a programului, notată cu N , atunci pentru etapa de procesare statică:

$$T_{static} = O(N)$$

Execuția poate deveni mai mare decât $O(N)$ dacă programul conține bucle care rulează de multe ori.

12 Complexitatea în spațiu

Spațiul folosit de engine-ul MS+ provine din mai multe structuri interne:

$$S_{total} = S_{tokens} + S_{AST} + S_{variabile} + S_{recursie} + S_{output}$$

Așadar, prin aproximare obținem:

$$S_{total} = O(m + k + v + h + o)$$

unde:

- m = numărul de token-uri
- k = numărul de noduri din AST
- v = numărul de variabile distincte
- h = înălțimea maximă a AST-ului
- o = numărul de valori afișate la output

De obicei, în cadrul programelor educaționale, output-ul și numărul de variabile sunt mici, iar spațiul este asimilat de token-uri și AST:

$$S_{total} \approx O(m + k)$$

Desigur, asta nu înseamnă că este un lucru fixat. Se pot realiza și programe complexe dpdv. al utilizării resurselor disponibile.

13 Despre Analizatorul de Complexitate

Analizatorul de Complexitate extinde capabilitatea engine-ului, evaluând prin aproximare complexitatea programului. Acesta nu execută programul pentru toate valorile posibile de input. În schimb, el estimează complexitatea pe baza structurii sintactice a codului. Sistemul identifică următoarele caracteristici:

- L = numărul total de bucle
- d = adâncimea maximă a buclelor imbricate
- v = numărul de variabile folosite
- b = numărul de blocuri condiționale

O estimare simplificată pentru complexitatea în timp este:

$$C_{time} \approx O(n^d)$$

unde d este adâncimea maximă a buclelor imbricate detectate. Putem observa în tabelul de mai jos exemplificat:

Structura programului	Complexitate estimată
fără bucle relevante	$O(1)$
o buclă simplă	$O(n)$
două bucle imbricate	$O(n^2)$
trei bucle imbricate	$O(n^3)$

Atunci când vine vorba de complexitatea în spațiu, analizatorul urmărește variabilele și structurile auxiliare:

$$C_{space} \approx O(v + a)$$

unde a reprezintă eventuale structuri auxiliare detectate. În versiunea educațională a MiniScript+, unde accentul este pus pe variabile simple și control de flux, estimarea spațiului este de obicei:

$$C_{space} = O(1)$$

dacă nu există structuri care cresc odată cu input-ul. Desigur, asta nu înseamnă că acesta va fi mereu constant.

14 Calcularea scorului de eficiență

Pentru a oferi feedback vizual, analizatorul poate transforma informațiile structurale într-un scor de eficiență astfel:

$$Score = 100 - \alpha L - \beta d - \gamma b$$

unde:

L = numărul de bucle

d = adâncimea maximă a buclelor

b = numărul de blocuri condiționale

α, β, γ = coeficienți de penalizare

Acest scor nu reprezintă o demonstrație formală a performanței, ci o aproximare la nivel educațional care ajută utilizatorul să observe dacă soluția are multe bucle sau blocuri imbricate.

15 Exemplu de analiză

Să luăm concret următorul program:

```
X = 0
WHILE X < N
  PRINT X
  X = X + 1
END
```

există o singură buclă, deci:

$$d = 1$$

Dacă bucla rulează de N ori, complexitatea în timp este:

$$T(N) = O(N)$$

Numărul de variabile folosite este constant, deci:

$$S(N) = O(1)$$

Pentru două bucle imbricate:

```
I = 0
WHILE I < N
  J = 0
  WHILE J < N
    PRINT I + J
    J = J + 1
  END
  I = I + 1
END
```

adâncimea maximă este:

$$d = 2$$

iar complexitatea estimată este:

$$T(N) = O(N^2)$$

16 Observații privind corectitudinea analizei

De ținut cont este faptul că **Analizatorul de Complexitate** oferă o estimare **euristică**. Acesta folosește structura codului, nu o demonstrație matematică completă pentru orice input posibil. De exemplu, o buclă de forma:

WHILE X < 10

poate avea un număr constant de iterații, deci complexitatea ei poate fi tratată ca:

$$O(1)$$

în timp ce o buclă dependentă de input:

WHILE X < N

este estimată ca:

$$O(n)$$

Prin urmare, scopul analizatorului este educațional: să ofere începătorului o aproximare clară a eficienței algoritmului și să evidențieze impactul buclelor, al imbricării și al structurii codului.

17 Concluzie

Engine-ul MiniScript+ poate fi descris matematic ca un interpretor bazat pe transformări succesive ale codului: de la procesul de tokenizare, parsare, construire AST până la evaluare. Fiecare instrucțiune modifică un state intern, iar expresiile sunt evaluate recursiv pe baza arborelui sintactic abstract.

Desigur, această abordare permite implementarea și extinderea funcționalității limbajului de programare: rularea programelor, debugging pas cu pas, evidențierea liniei curente și estimarea complexității algoritmului. Astfel, **MiniScript+** nu este doar un **limbaj educațional simplificat**, ci și un model practic pentru înțelegerea modului în care funcționează un interpretor.